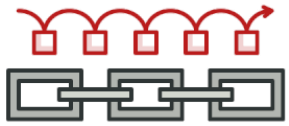
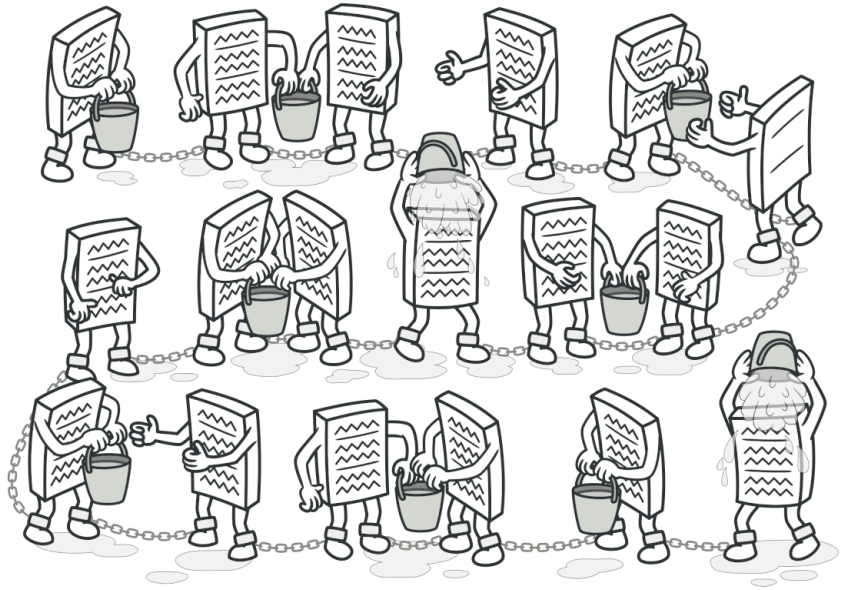
Поведенческие паттерны. Цепочка обязанностей

Поведенческие паттерны:

- **Определяют** алгоритмы и способы реализации взаимодействия различных объектов и классов в программе.
- **Отвечают** за управление различными аспектами поведения объектов и обеспечение гибкости и расширяемости системы.



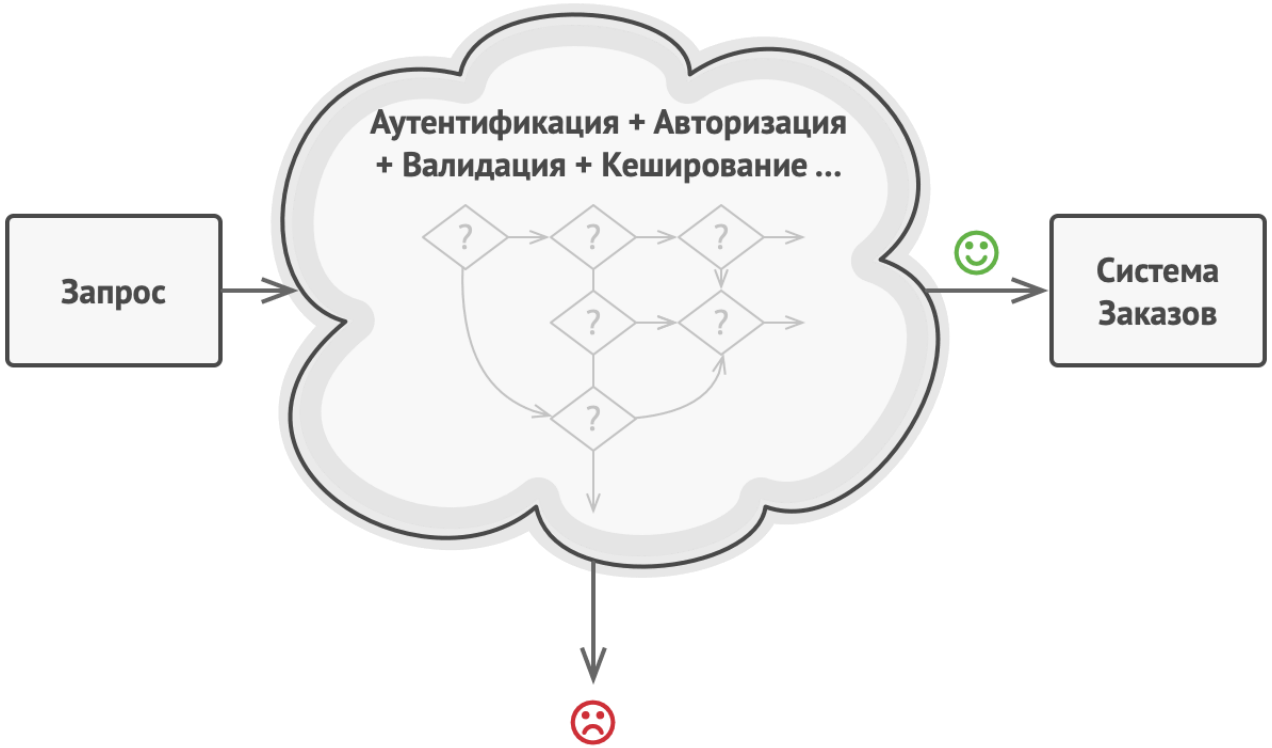
Паттерн Цепочка обязанностей
Chain of Responsibility (Chain of Command)



Цепочка обязанностей — это поведенческий паттерн, позволяющий передавать запрос по цепочке потенциальных обработчиков, пока один из них не обработает запрос.

Избавляет от жёсткой привязки отправителя запроса к его получателю, позволяя выстраивать цепь из различных обработчиков динамически.

Проблема



Наличие большого количества взаимосвязанных проверок усложняют модификацию модуля из-за необходимости согласований при изменении даже одного условия

Решение

Цепочка обязанностей **базируется на том, чтобы превратить отдельные поведения в объекты.**
Каждая проверка будет реализована в виде отдельного класса с единственным методом выполнения. Данные запроса, над которым происходит проверка, будут передаваться в метод как аргументы.



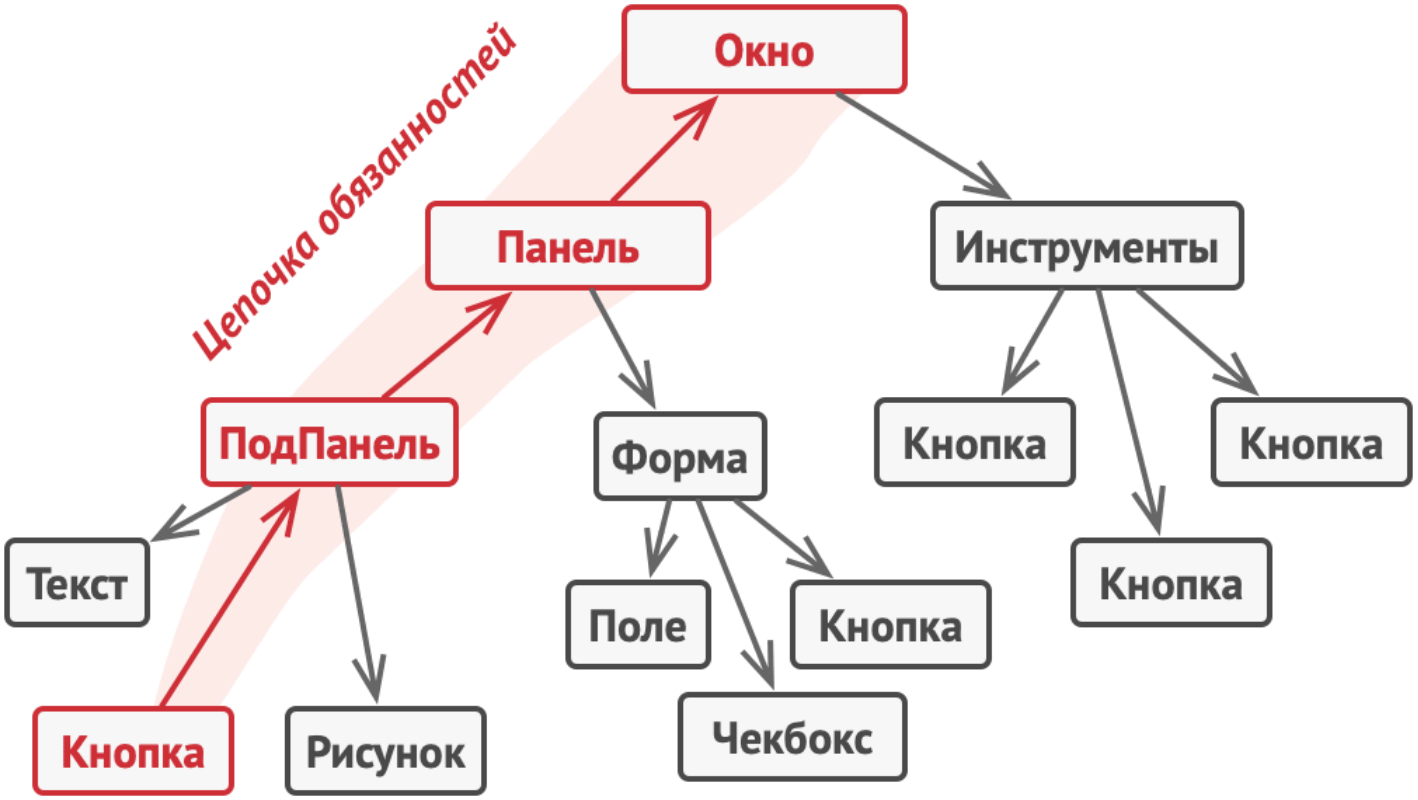
- Паттерн предлагает связать объекты обработчиков в одну цепь. Каждый из них будет иметь ссылку на следующий обработчик в цепи. При получении запроса обработчик сможет не только сам выполнить действия, но и передать обработку следующему объекту в цепочке.
- Все объекты в цепи смогут обработать запрос. Длина цепочки не имеет значения.
- Обработчик не обязательно должен передавать запрос дальше, он может прервать процедуру.

Решение

Обработчики прерывают цепь когда они *могут* обработать запрос или при возникновении определенной ситуации.

Запрос движется по цепи, пока не найдётся обработчик, который может его обработать.

Все объекты цепочки должны иметь общий интерфейс

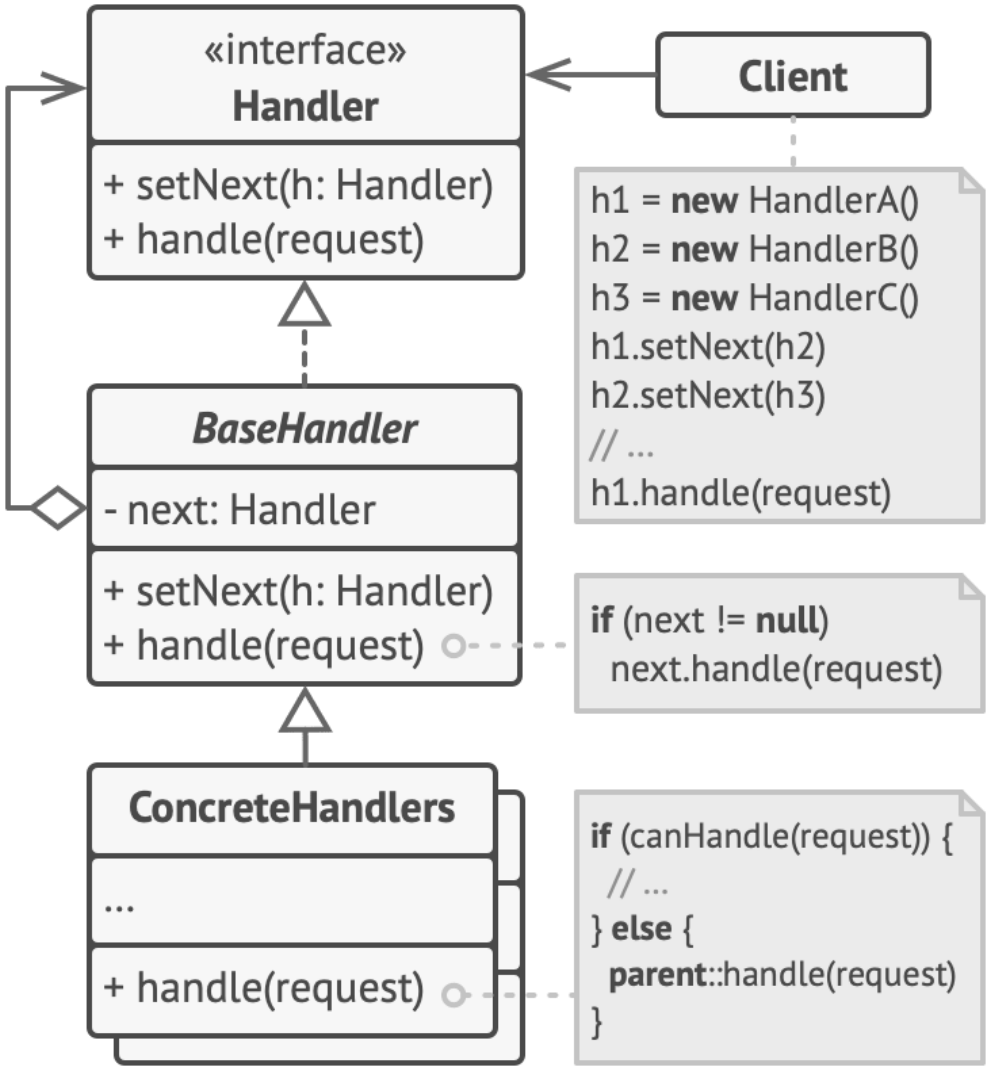


Структура паттерна

[1]**Обработчик** определяет общий для всех конкретных обработчиков интерфейс. Обычно достаточно описать единственный метод обработки запросов, но иногда здесь может быть объявлен и метод выставления следующего обработчика.

[2]**Базовый обработчик** — опциональный класс, который позволяет избавиться от дублирования одного и того же кода во всех конкретных обработчиках.

Обычно этот класс имеет поле для хранения ссылки на следующий обработчик в цепочке. Клиент связывает обработчики в цепь, подавая ссылку на следующий обработчик через конструктор или сеттер поля. Также здесь можно реализовать базовый метод обработки, который бы просто перенаправлял запрос следующему обработчику, проверив его наличие.

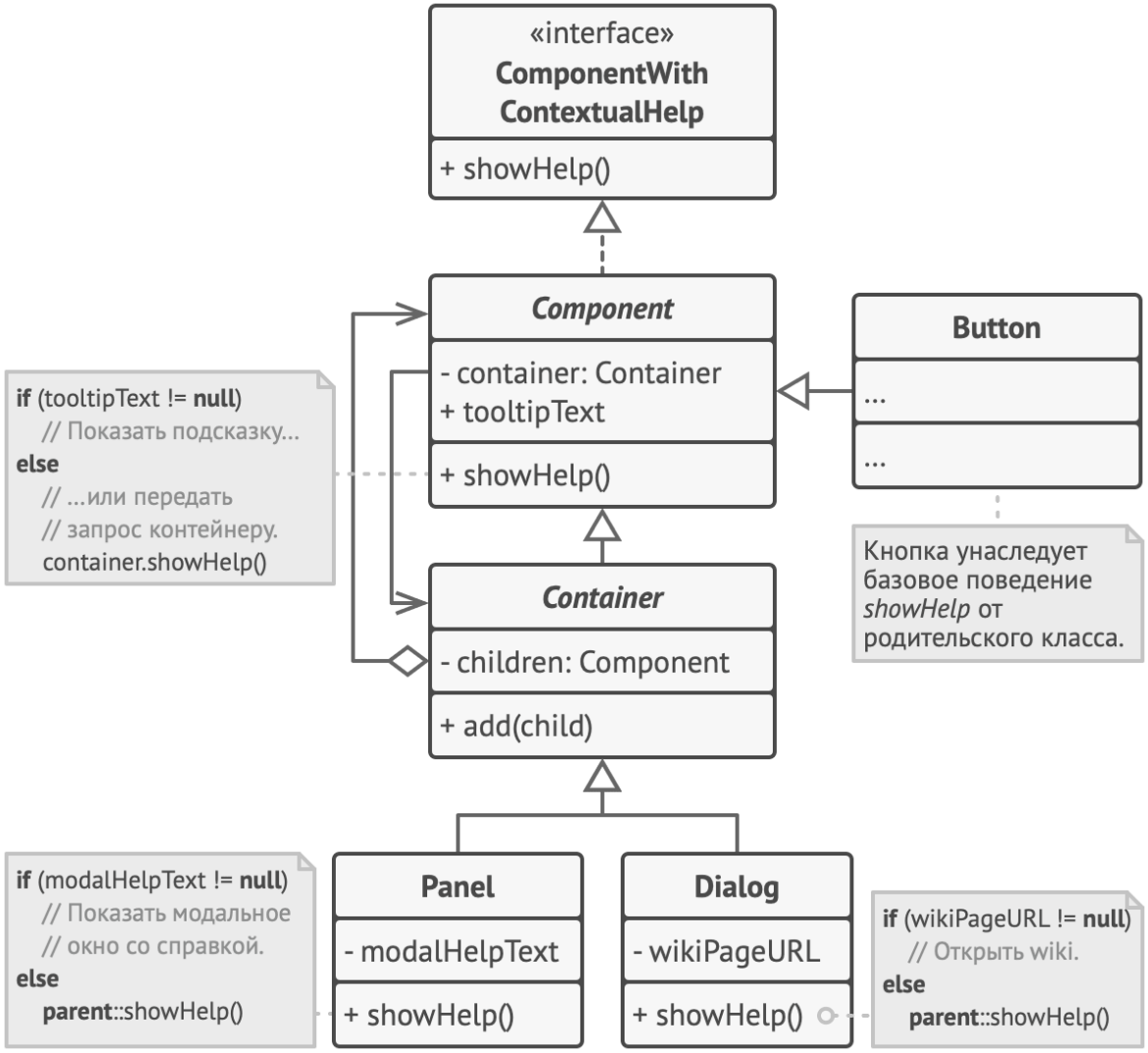


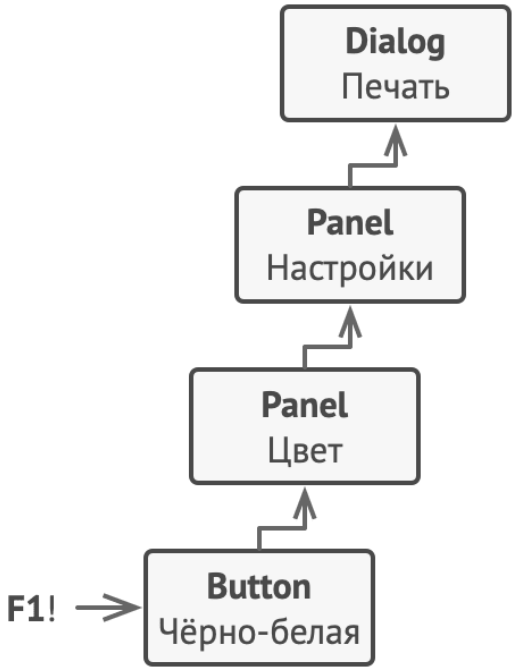
[4]**Клиент** может либо сформировать цепочку обработчиков единожды, либо перестраивать её динамически, в зависимости от логики программы. Клиент может отправлять запросы любому из объектов цепочки, не обязательно первому из них.

[3]**Конкретные обработчики** содержат код обработки запросов. При получении запроса каждый обработчик решает, может ли он обработать запрос, а также стоит ли передать его следующему объекту.
В большинстве случаев обработчики могут работать сами по себе и быть неизменяемыми, получив все нужные детали через параметры конструктора.

Пример

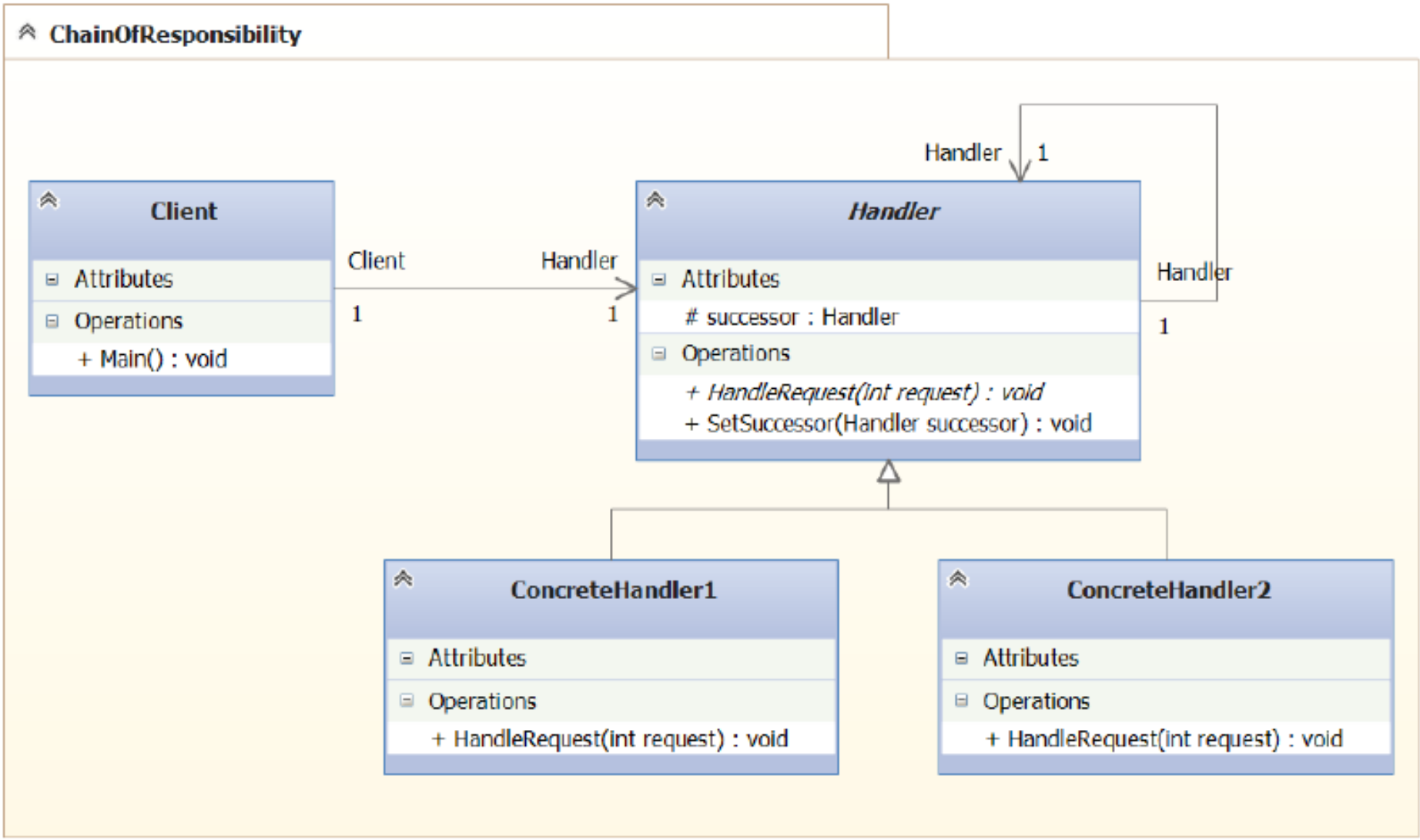
Цепочка обязанностей отвечает за показ контекстной помощи для активных элементов пользовательского интерфейса.





Вызова контекстной помощи в цепочке объектов UI. Запрос продвигается по списку контейнеров элемента, пока не находится тот, который способен отобразить помощь.

Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        Handler h1 = new ConcreteHandler1();
        Handler h2 = new ConcreteHandler2();

        h1.Successor = h2;
        h1.HandleRequest(1);
        h1.HandleRequest(2);
    }
}
```

`Client` - **Клиент**:
Посылает запрос определенному объекту класса `ConcreteHandler`, который находится в цепочке объектов-обработчиков.

`Handler` - **Обработчик**:
Предоставляет интерфейс для обработки запросов.

```
abstract class Handler
{
    public Handler Successor { get; set; }
    public abstract void HandleRequest(int request);
}
```

`ConcreteHandler` - **Конкретный обработчик**:
Обрабатывает запрос, который предназначен ему. Имеет ссылку на своего преемника, которому передает запрос в случае, если сам не в состоянии его обработать.

```
class ConcreteHandler1 : Handler
{
    public override void HandleRequest(int request)
    {
        if (request == 1)
            Console.WriteLine("One");
        else if (Successor != null)
            Successor.HandleRequest(request);
    }
}
```

```
class ConcreteHandler2 : Handler
{
    public override void HandleRequest(int request)
    {
        if (request == 2)
            Console.WriteLine("Two");
        else if (Successor != null)
            Successor.HandleRequest(request);
    }
}
```

Отношения между участниками

Отношения между классами

- Абстрактный класс Handler связан связью отношения самоассоциации.
- Конкретные классы ConcreteHandler1 и ConcreteHandler2 связаны связью отношения наследования с абстрактным классом Handler.
- Отношения между объектами
- Когда клиент посылает запрос, этот запрос передается по цепочке, пока один из объектов ConcreteHandler не обработает этот запрос.



Применение

Когда программа должна обрабатывать разнообразные запросы несколькими способами, но заранее неизвестно, какие конкретно запросы будут приходить и какие обработчики для них понадобятся.

С помощью *Цепочки обязанностей* можно связать потенциальных обработчиков в одну цепь и при получении запроса поочерёдно обращаться к каждому из них для обработки запроса.

Когда важно, чтобы обработчики выполнялись один за другим в строгом порядке.

Цепочка обязанностей позволяет запускать обработчики последовательно один за другим в том порядке, в котором они находятся в цепочке.

Когда набор объектов, способных обработать запрос, должен задаваться динамически.

В любой момент можно модифицировать существующую цепочку, переназначив связи так, чтобы убрать или добавить новое звено.

Шаги реализации

1. Создать интерфейс обработчика и описать в нём основной метод обработки.

Рекомендуется преобразовать данные запроса в объект и передавать его целиком через параметры метода обработчика.

2. При необходимости создать абстрактный базовый класс обработчиков, чтобы не дублировать реализацию метода получения следующего обработчика во всех конкретных обработчиках.

Добавить в базовый обработчик поле для хранения ссылки на следующий объект цепочки. Начальное значение этого поля устанавливать через конструктор. Это сделает объекты обработчиков неизменяемыми. Если программа предполагает динамическую перестройку цепочек, можно добавить сеттер для поля.

Реализовать базовый метод обработки так, чтобы он перенаправлял запрос следующему объекту, проверив его наличие. Это позволит полностью скрыть поле-ссылку от подклассов, дав им возможность передавать запросы дальше по цепи, обращаясь к родительской реализации метода.

3. Создать классы конкретных обработчиков и реализовать в них методы обработки запросов. При получении запроса каждый обработчик должен решить:

- Может ли он обработать запрос или нет?
- Следует ли передать запрос следующему обработчику или нет?

Шаги реализации

4. **Клиент может собирать цепочку обработчиков самостоятельно**, опираясь на свою бизнес-логику, либо получать уже готовые цепочки извне. В последнем случае цепочки собираются фабричными объектами, опираясь на конфигурацию приложения или параметры окружения.

5. Клиент может посылать запросы любому обработчику в цепи, а не только первому. Запрос будет передаваться по цепочке до тех пор, пока какой-то обработчик не откажется передавать его дальше, либо когда будет достигнут конец цепи.

6. Клиент должен знать о динамической природе цепочки и быть готов к таким случаям:

- Цепочка может состоять из единственного объекта.
- Запросы могут не достигать конца цепи.
- Запросы могут достигать конца, оставаясь необработанными.

Преимущества и недостатки

+ Уменьшает *зависимость* между клиентом и обработчиками.
+ Реализует принцип *единственной обязанности*.
+ Реализует принцип *открытости/закрытости*.
- Запрос может остаться никем не обработанным.